



ORACLE

MySQL Implementation Essentials Bootcamp

High Availability

Marco Carlessi (marco.carlessi@oracle.com)
Andreea Ciobotaru (andreea.Ciobotaru@oracle.com)
MySQL Sales Consulting

2023



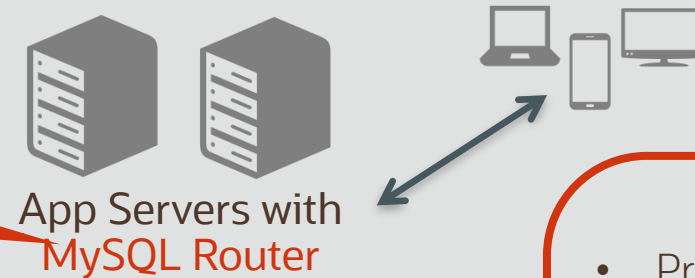
Program Agenda

- Introduction
- Oracle MySQL Enterprise
- Oracle Cloud Infrastructure overview
- Installation and Architecture
- Database Design
- Security
- Backup
- Replication
- **High Availability**
- MySQL Enterprise Monitor
- HeatWave

MySQL InnoDB Cluster

“A single product — MySQL — with high availability and scaling features baked in; providing an integrated end-to-end solution that is easy to use.”

- Route application connection to available nodes
- Automatic configuration
- REST API interface for monitoring



- Automate cluster creation and operation (clone)
- scriptable (JavaScript, Python, SQL)
- Document Store scripting

MySQL Shell
Setup, Manage,
Orchestrate

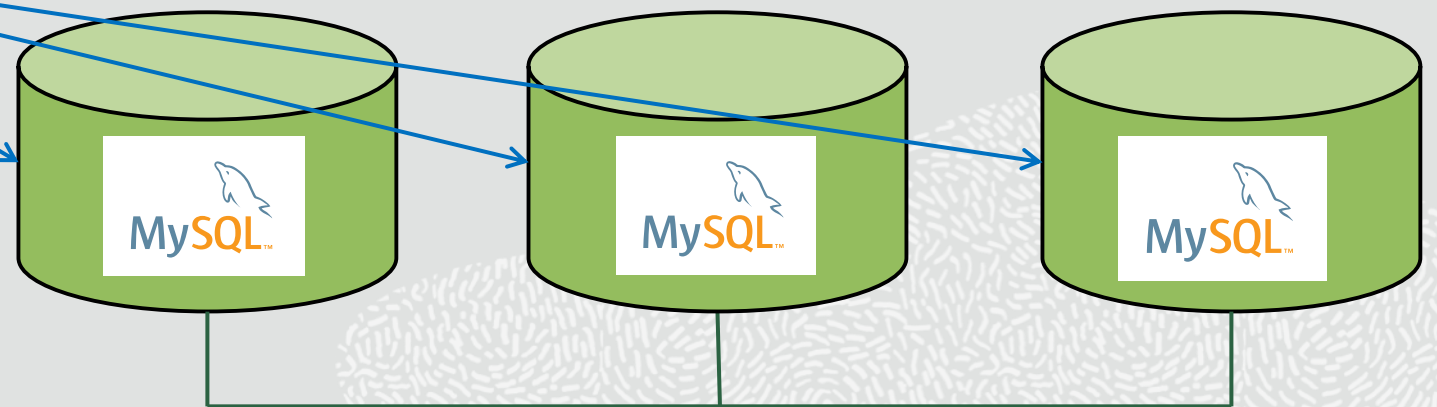
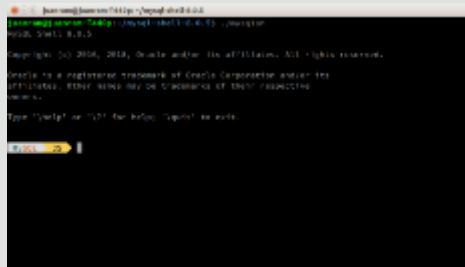
- Provides virtually synchronous replication
 - with Consistent Reads
- Automate operations
 - Conflict detection and resolution
 - Failure detection, fail-over, recovery
 - Group membership management and creation

MySQL Server w/ Group Replication plugin

Available on all MySQL supported platforms

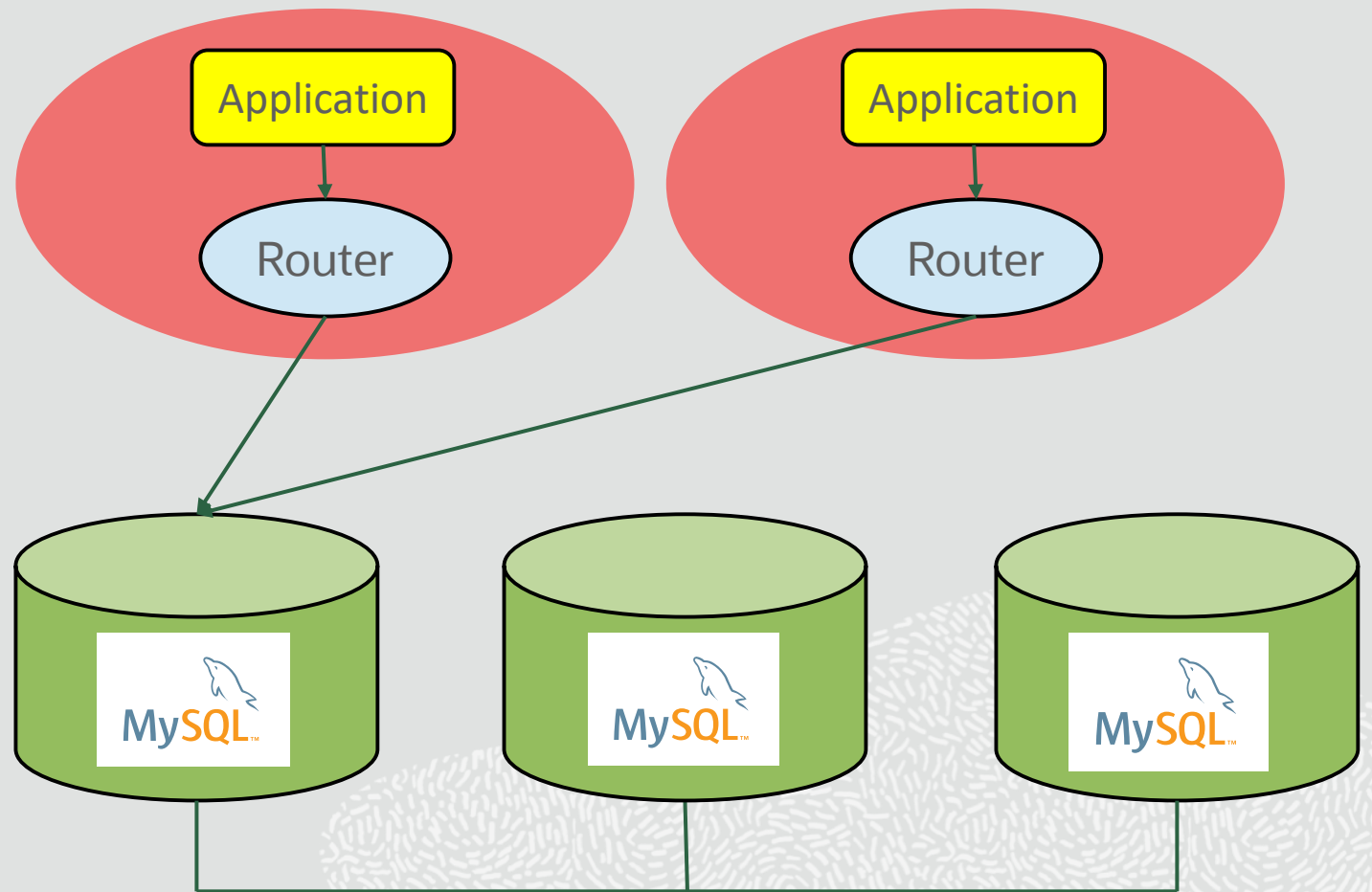
InnoDB Cluster: Setup

- Manage of the cluster it's easy with MySQL Shell: creation, status check, configuration changes... and cloning!
- Setup a cluster with a remote connection to 3 instances



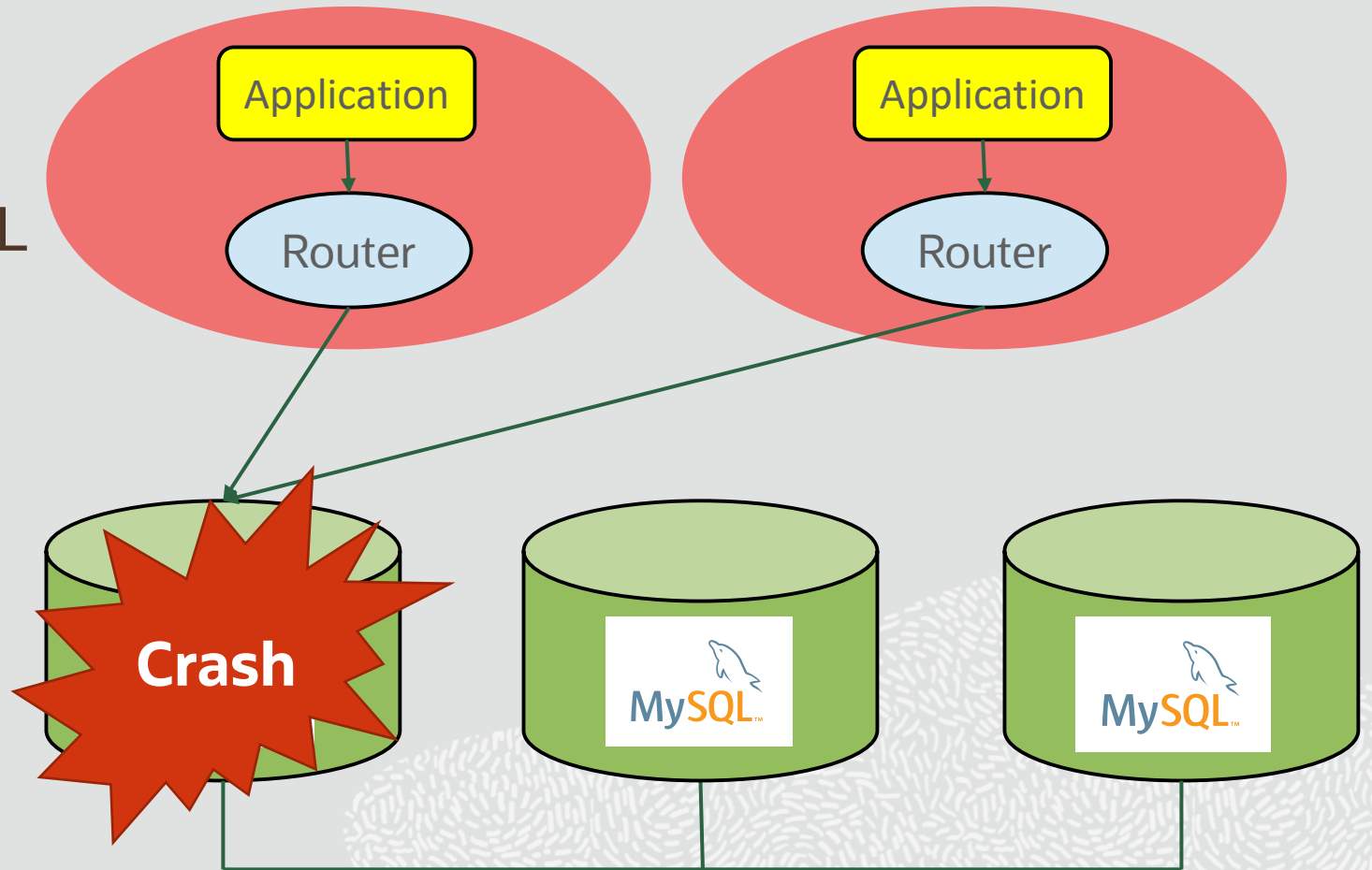
InnoDB Cluster: Application access

- Start Router with 'bootstrap' option that automatically create Router configuration



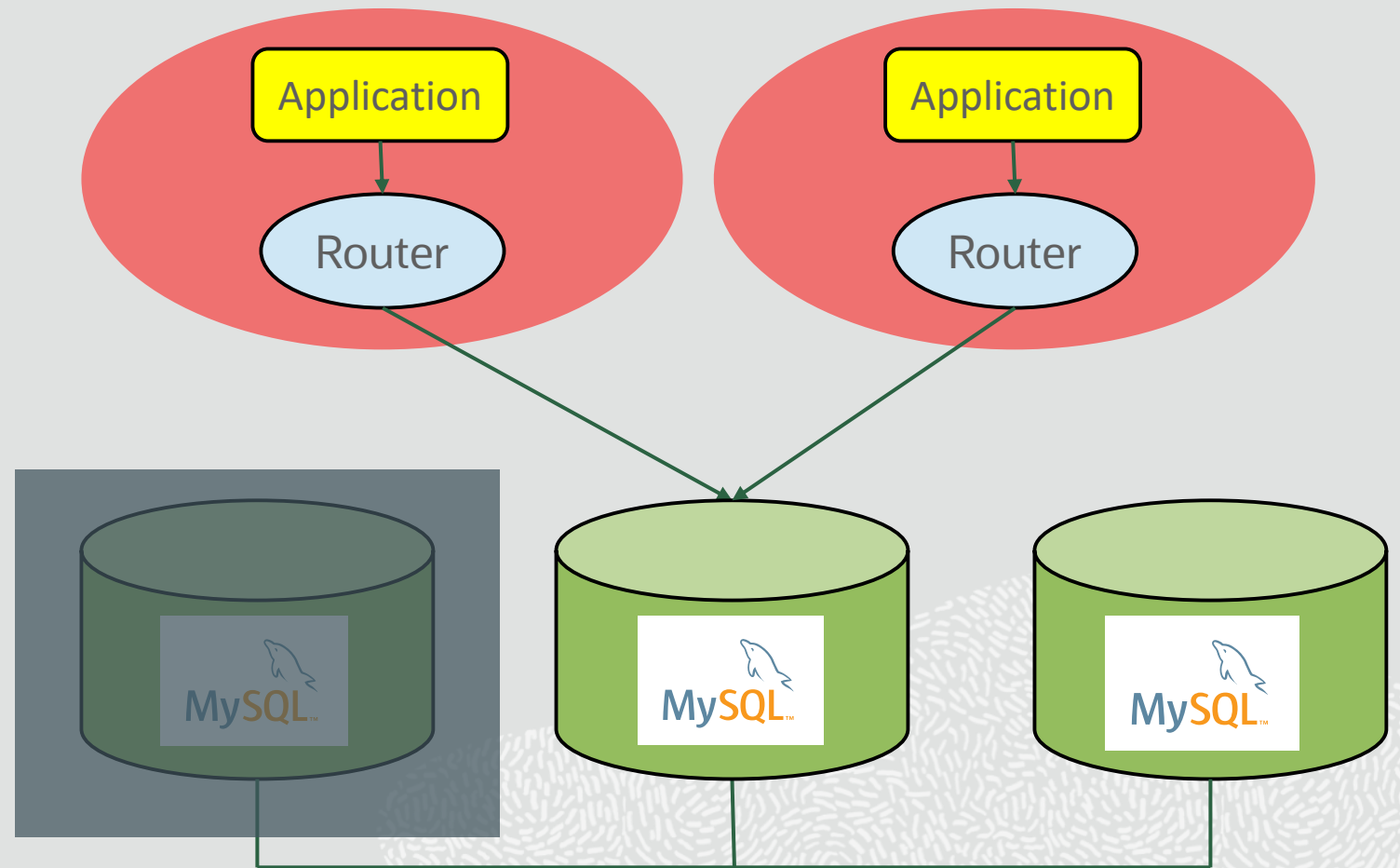
InnoDB Cluster: Application access

- Application accesses the database through MySQL Router
- In case of failure...



InnoDB Cluster: Application access

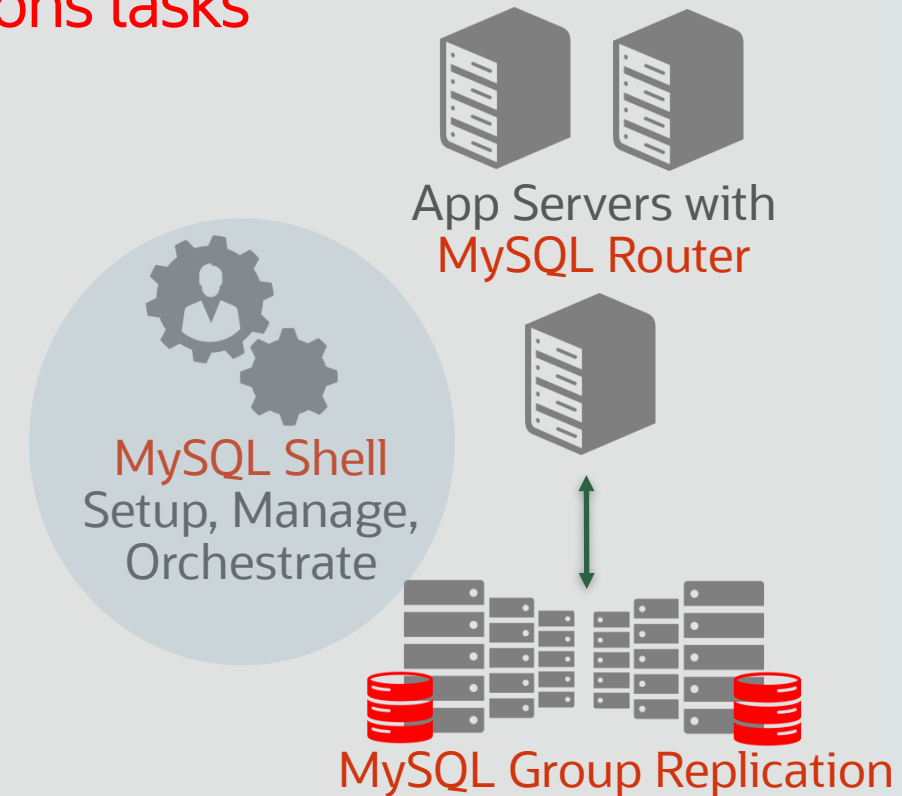
- ...connections are routed to an available node
- No need of VIP or configuration changes to the application



MySQL Shell

A single unified client for all administrative and operations tasks

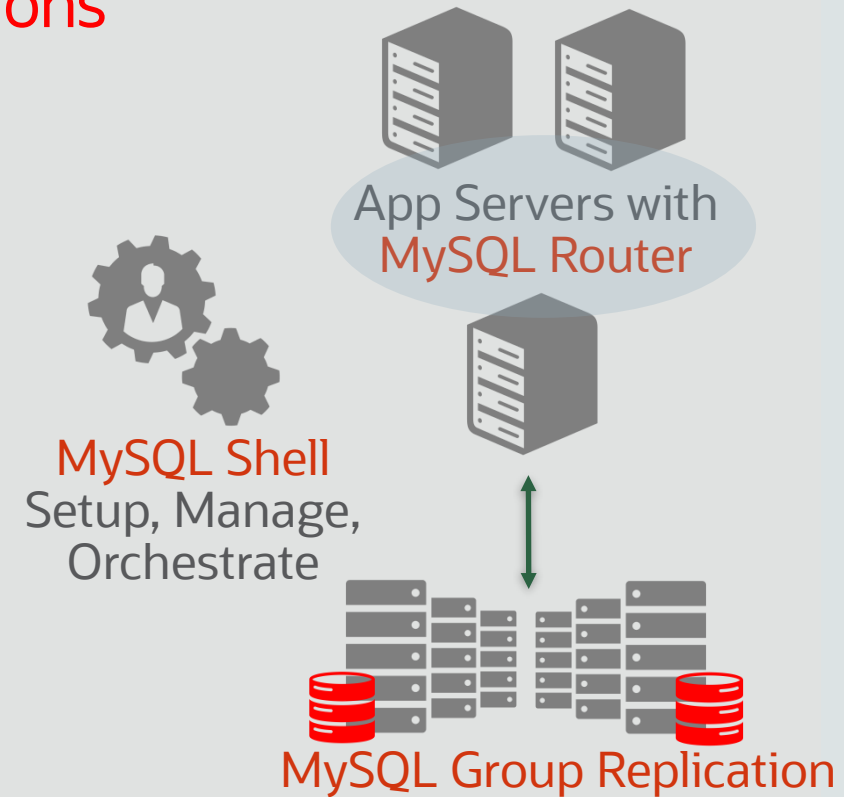
- Multi-Language: JavaScript, Python, and SQL
 - Naturally scriptable
- Supports both Document and Relational models
- Exposes full Development and Admin API
- Perform DBA operations
- Create and Manage MySQL InnoDB clusters, ReplicaSet and ClusterSet



MySQL Router

Transparent access to HA databases for MySQL Applications

- Transparent client connection routing
 - Load balancing
 - Application connection failover
- Stateless design offers easy HA client routing
 - A local Router becomes part of the application stack (installed in every application server)
 - Can be installed on dedicated servers
 - But HA have to be provided by 3rd party SW



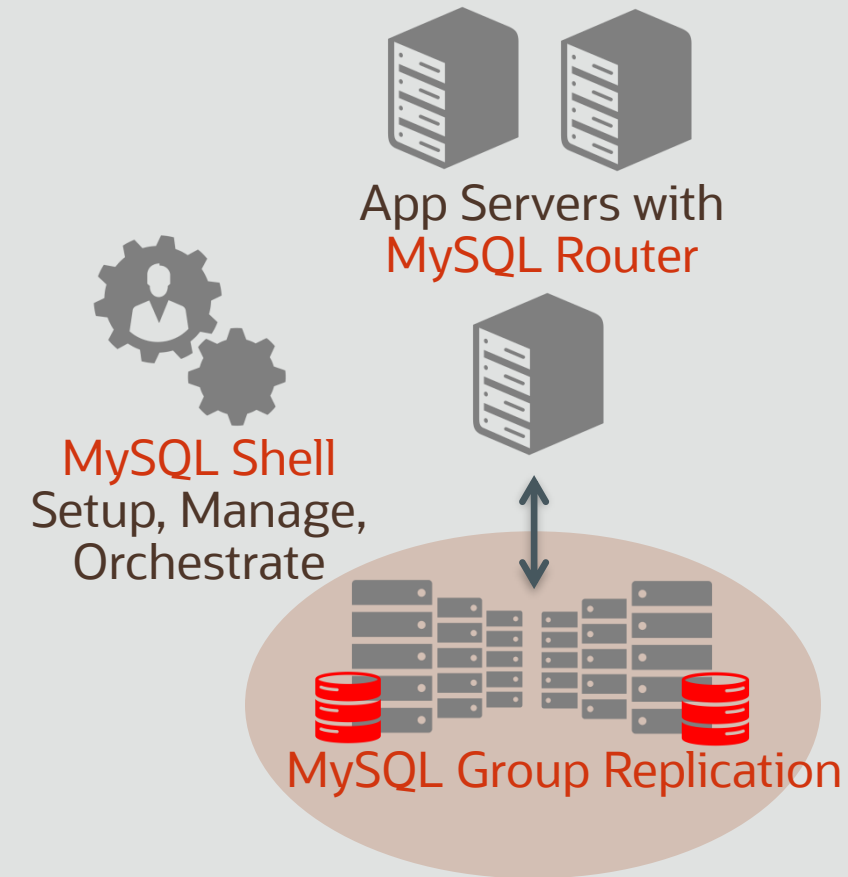
MySQL Automatic configuration

- Automatic configuration ("--bootstrap")
 - it may create a folder with settings, logs and startup/shutdown scripts
 - 2 ports for write (defaults: 6446 for sql client and 6448 for the new X protocol)
 - all connections are directed to primary
 - 2 ports for read (defaults: 6447 for sql client and 6449 for the new X protocol)
 - all connection are directed in round-robin to secondary only
- Script to start and stop are automatically created
- A user ('mysql_router...') for connection to cluster metadata is automatically generated

MySQL Group Replication: Database HA

- Group Replication library
 - Provides *virtually* synchronous replication for MySQL 5.7+
 - Supported on *all MySQL platforms*
 - Linux, Windows, Solaris, OSX, FreeBSD
 - Automates operations
 - Conflict detection and resolution
 - Failure detection, fail-over, recovery
 - Group membership management and reconfiguration

*“Multi-source **update anywhere** replication plugin for MySQL with built-in **conflict detection and resolution, automatic distributed recovery, and group membership.**”*



Hardware/Configuration Basics

- All servers should be equal
- Isolated machine resources
- Number of servers in the group must be 3, 5, 7 or 9
- Use SSD's if possible
- Redundancy
- Dual power supplies
- Dual network cards
 - Network – low latency and enough bandwidth
 - Maximize RAM

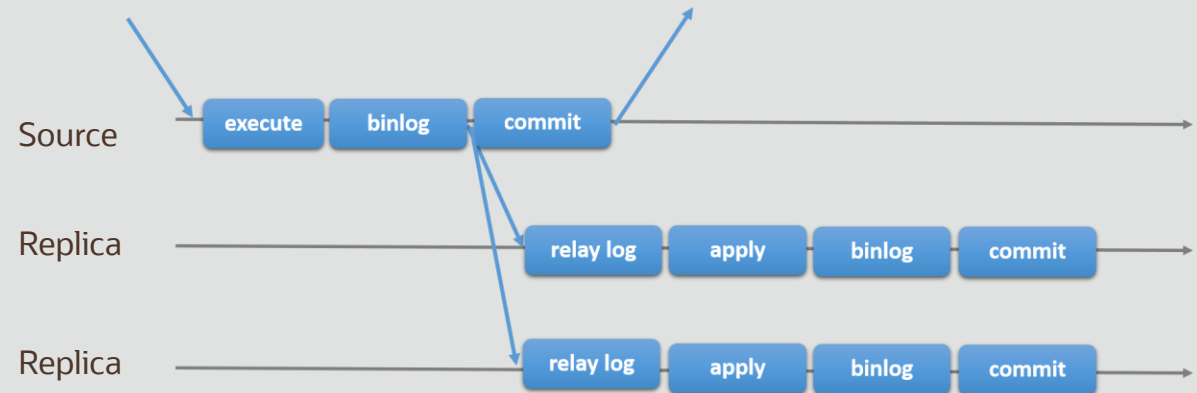
Group Replication: requirements

- MySQL 8.0
- Support for InnoDB only
- Primary key is required on every table
 - 8.0.23+: invisible columns may have Primary Key/Unique Index
 - 8.0.30+: GIPK mode automatically adds a generated invisible primary key (GIPK) to the tables created without an explicit primary key
- Binary Logs active (ROW)
- Global Transaction Identifiers (GTID) turned on
- Resolvable FQDN from each node in the cluster
- Full list here
 - <https://dev.mysql.com/doc/refman/8.0/en/group-replication-requirements.html>

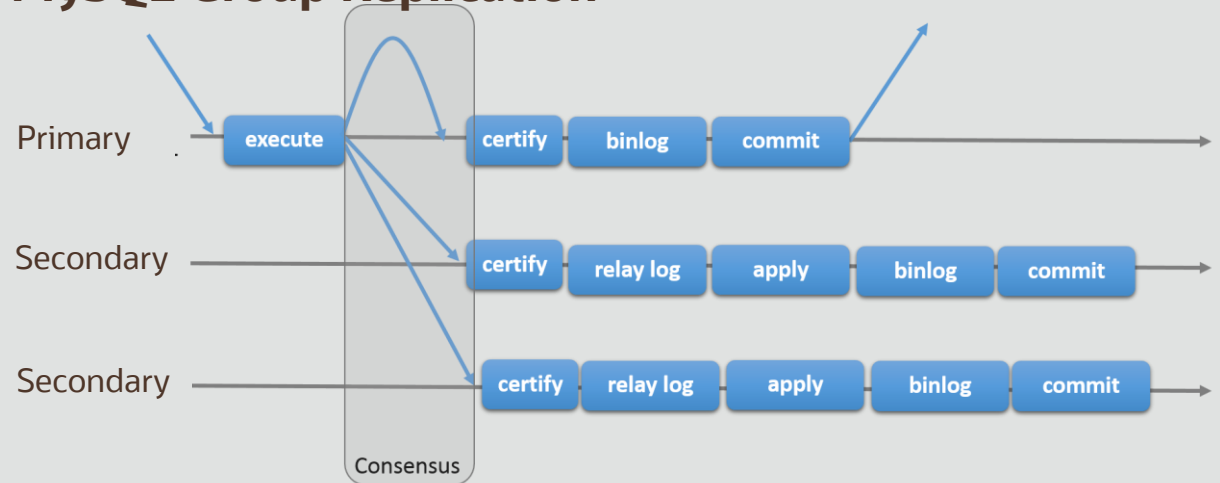
Group Replication

- Communication layer provides
 - Atomic messaging
 - Total order message delivery
- **Multi-Source update everywhere**
 - R/W transactions commit after group approval
 - Conflicts are auto-detected and managed (first-in wins)
- Very Large Transactions which are large enough that it cannot be copied between group members over the network within a 5 second window can cause failures in the group communication

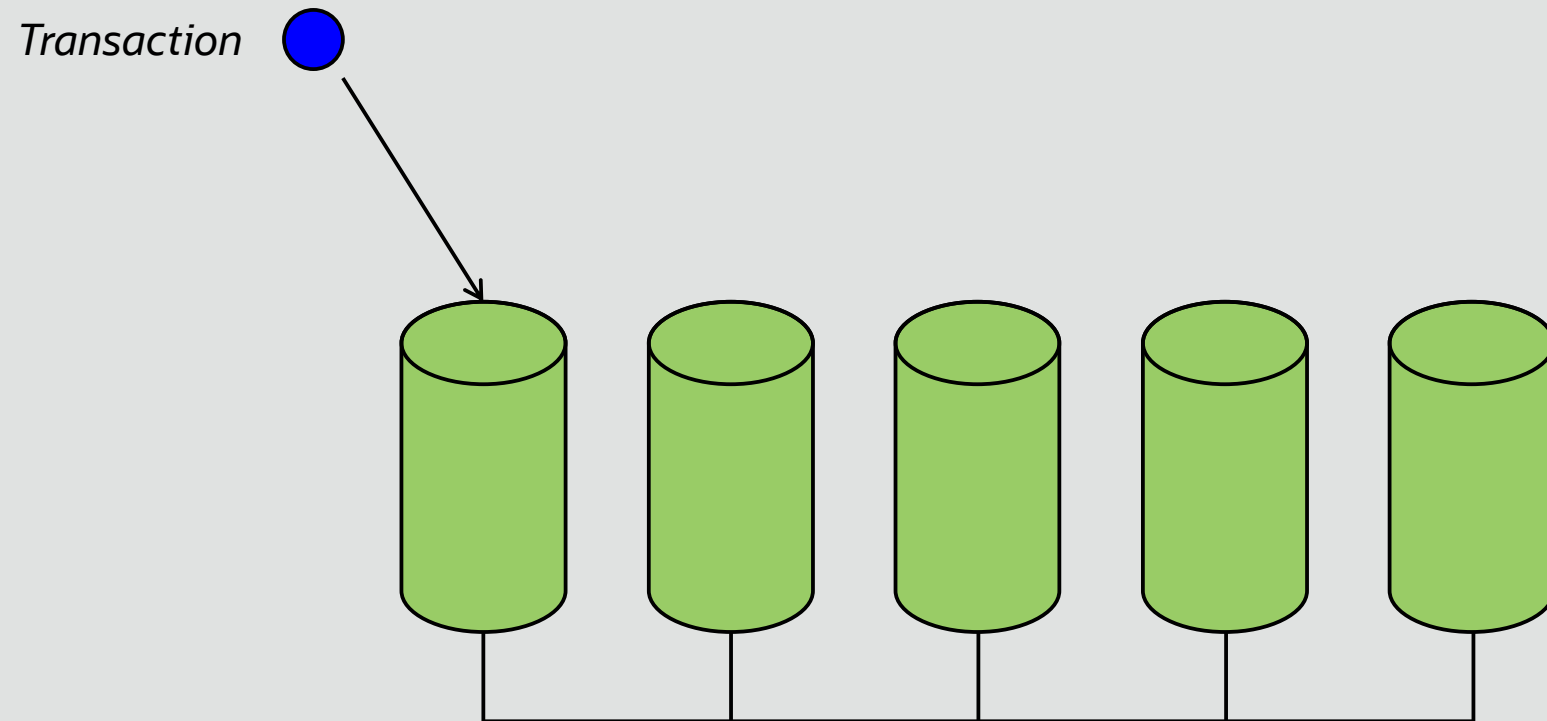
MySQL Asynchronous Replication



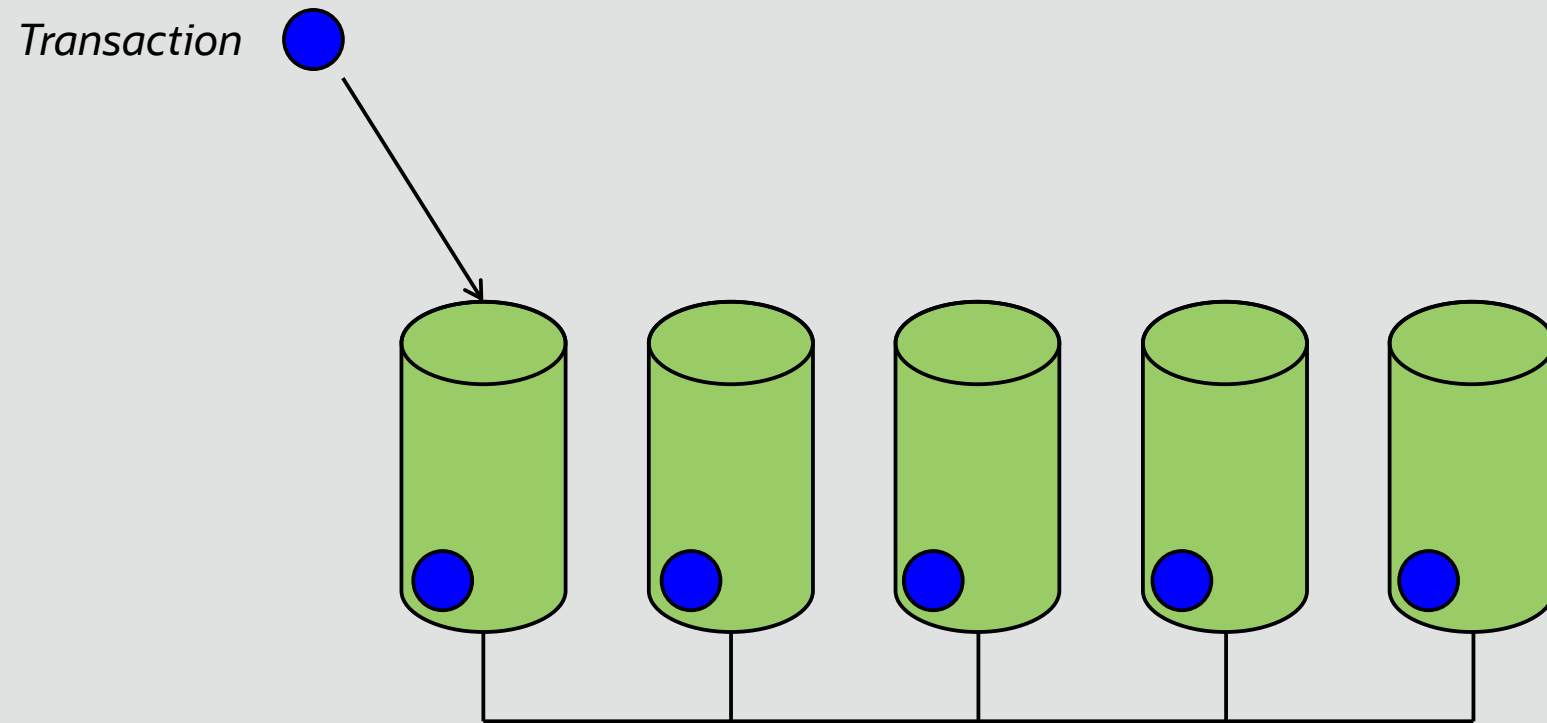
MySQL Group Replication



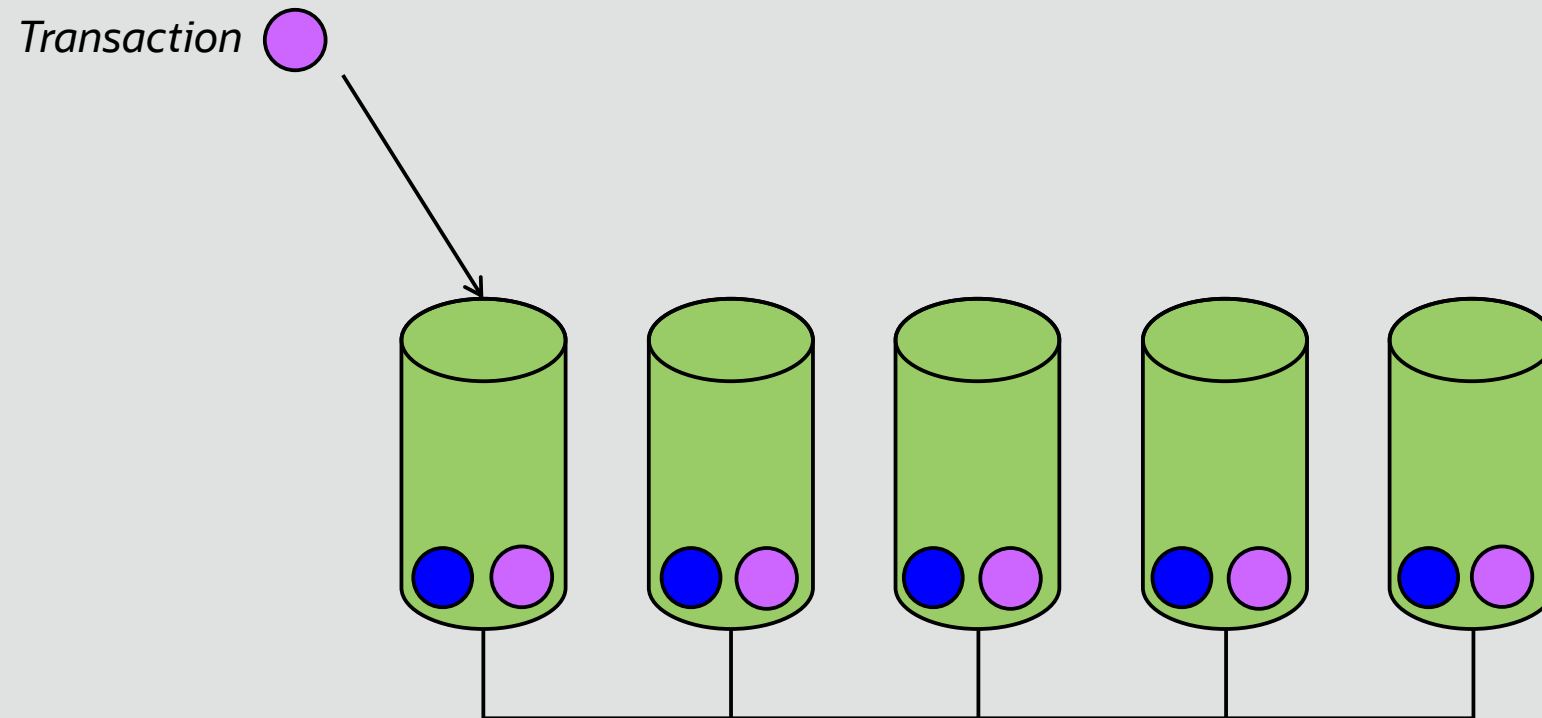
Split brain scenario: working cluster



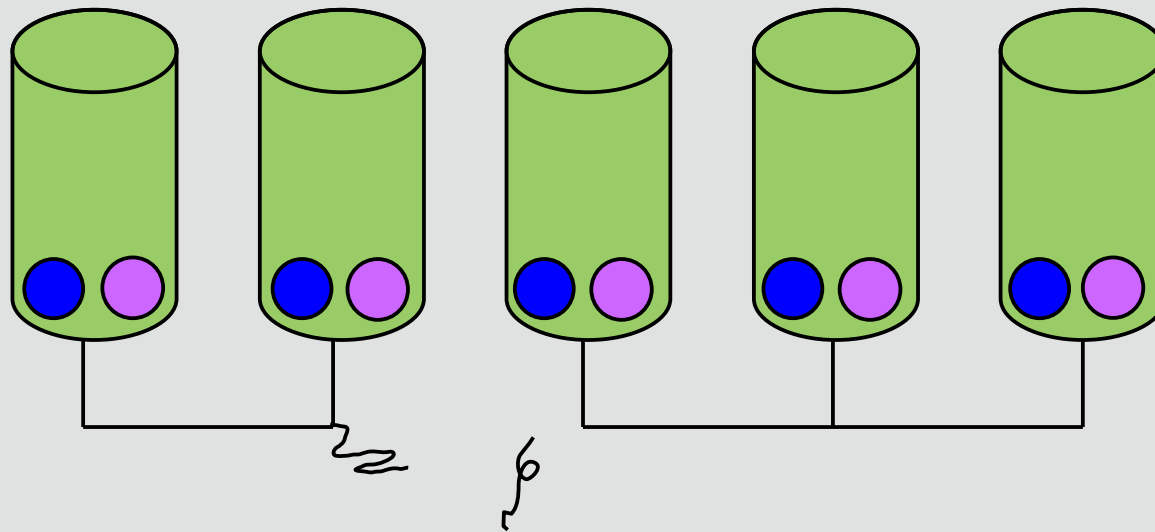
Split brain scenario: working cluster (contd.)



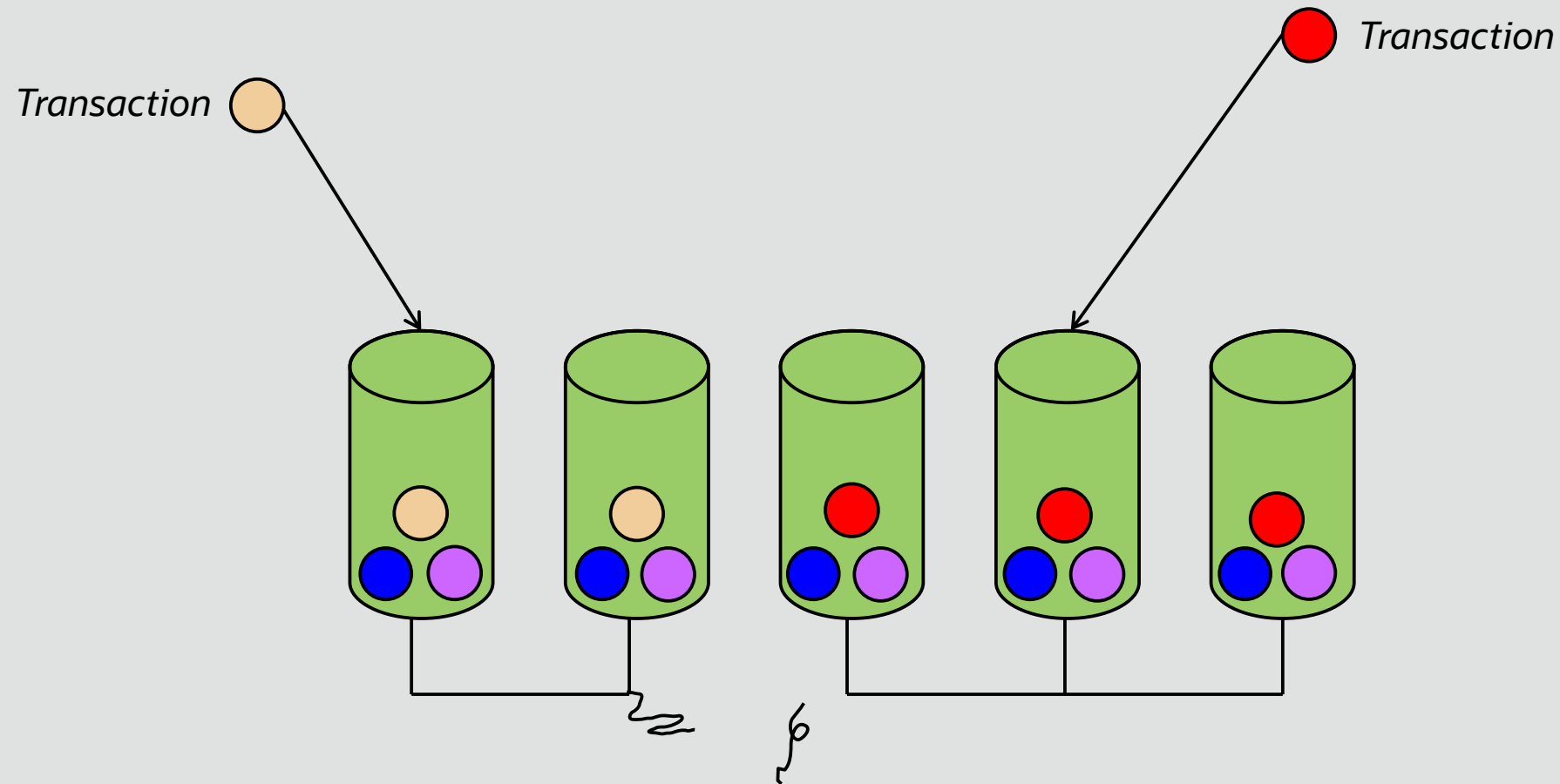
Split brain scenario: working cluster (contd.)



Split brain scenario : network split

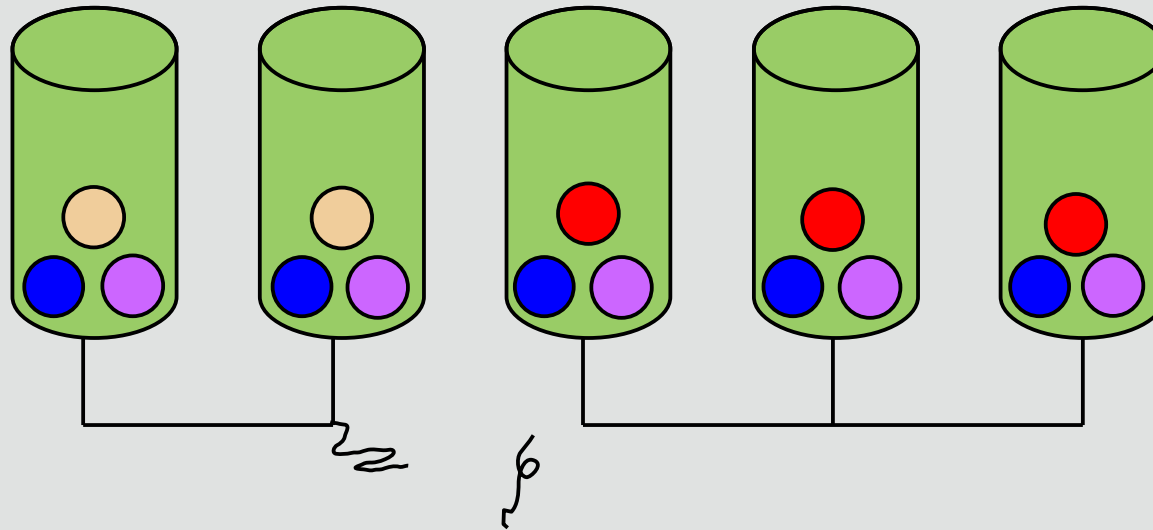


Split brain scenario: what happen if we do nothing?

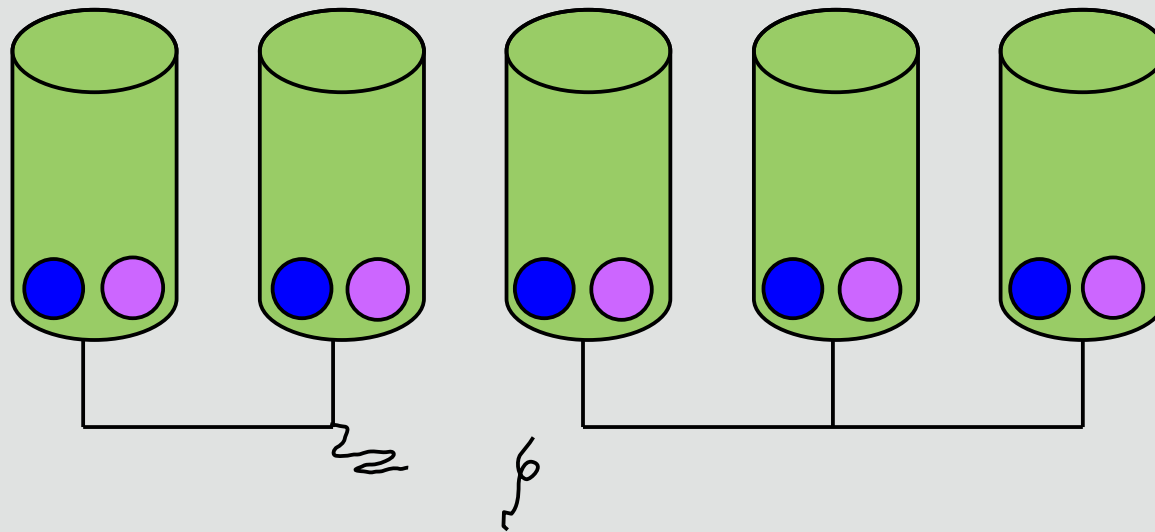


Split brain scenario: what happen if we do nothing?

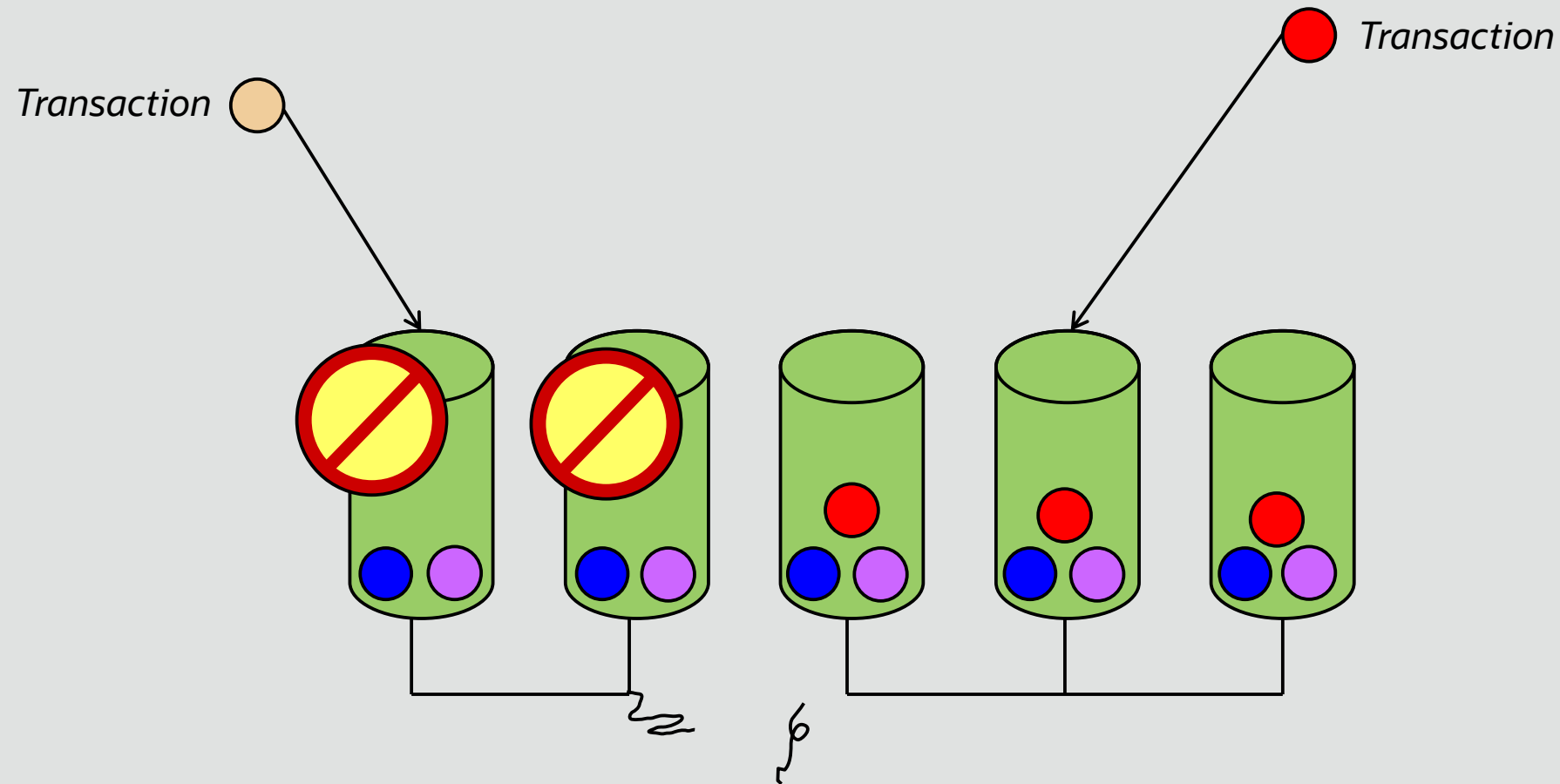
This is split-brain!
Inconsistent states!



Group Replication: network split



Group Replication: Split Brain prevention



Network Partitioning

- Requires a majority of members (consensus) to agree on a given decision
 - Paxos distributed algorithm
 - $n = (2 * f) + 1$
 - n = number of servers
 - f = number of failures
- A loss of majority of members results in shutdown
- Single-server exits result in group reconfiguration
- Plan for number of expected or anticipated possible failures
- Minimum number in group = 3
- Maximum number in group = 9

Group Size (n)	Majority	Failures Tolerated (f)
1	1	0
2	2	0
3	2	1
4	3	1
5	3	2
6	4	2
7	4	3
8	5	3
9	5	4

Monitoring Group Replication

- MySQL Enterprise Monitor
 - Automatic discover of InnoDB Clusters
 - Dedicated Advisors
 - Automatic update in case of topology changes

- Performance Schema tables

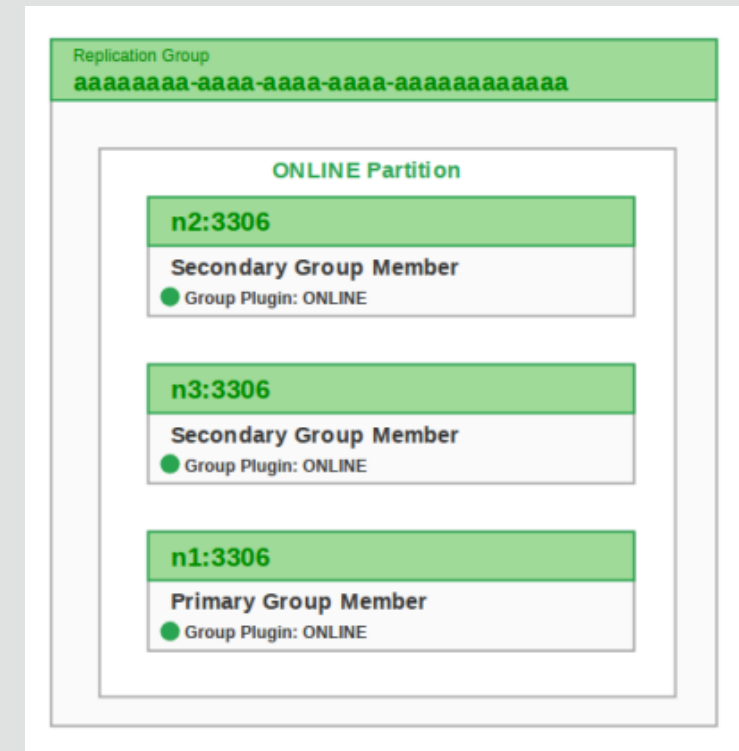
`performance_schema.replication_group_member_stats`

`performance_schema.replication_group_members`

`performance_schema.replication_connection_status`

`performance_schema.replication_applier_status`

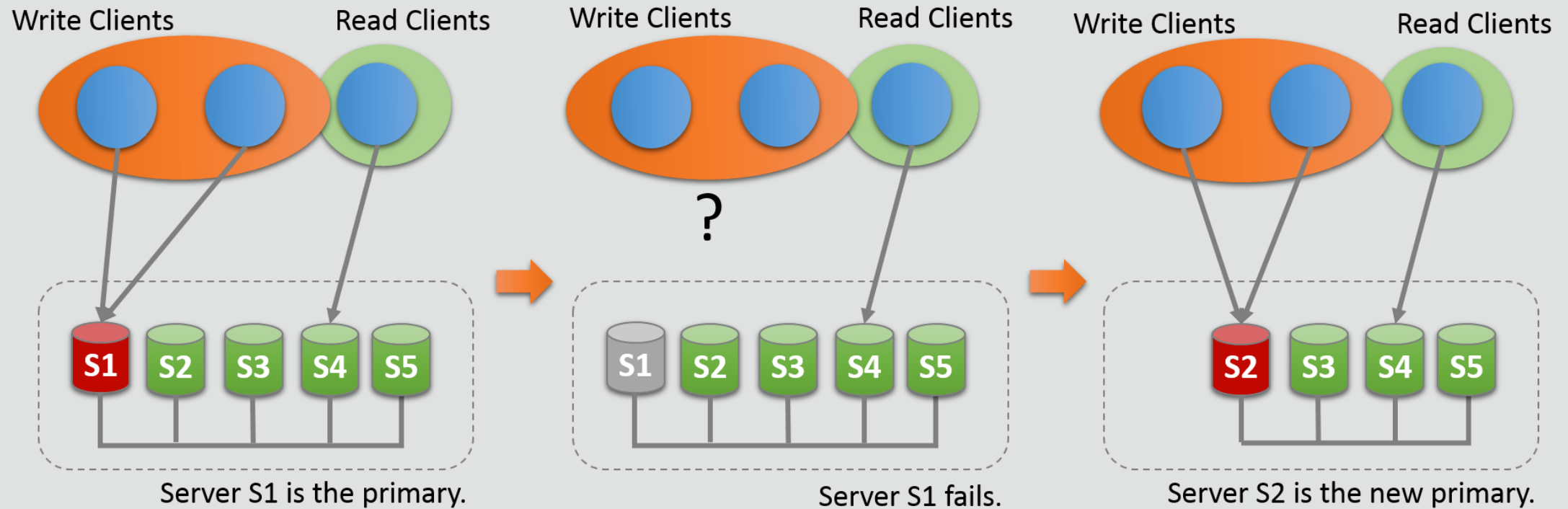
- Replication channels created:
 - `group_replication_recovery` - changes to replication
 - `group_replication_applier` - applies incoming transactions



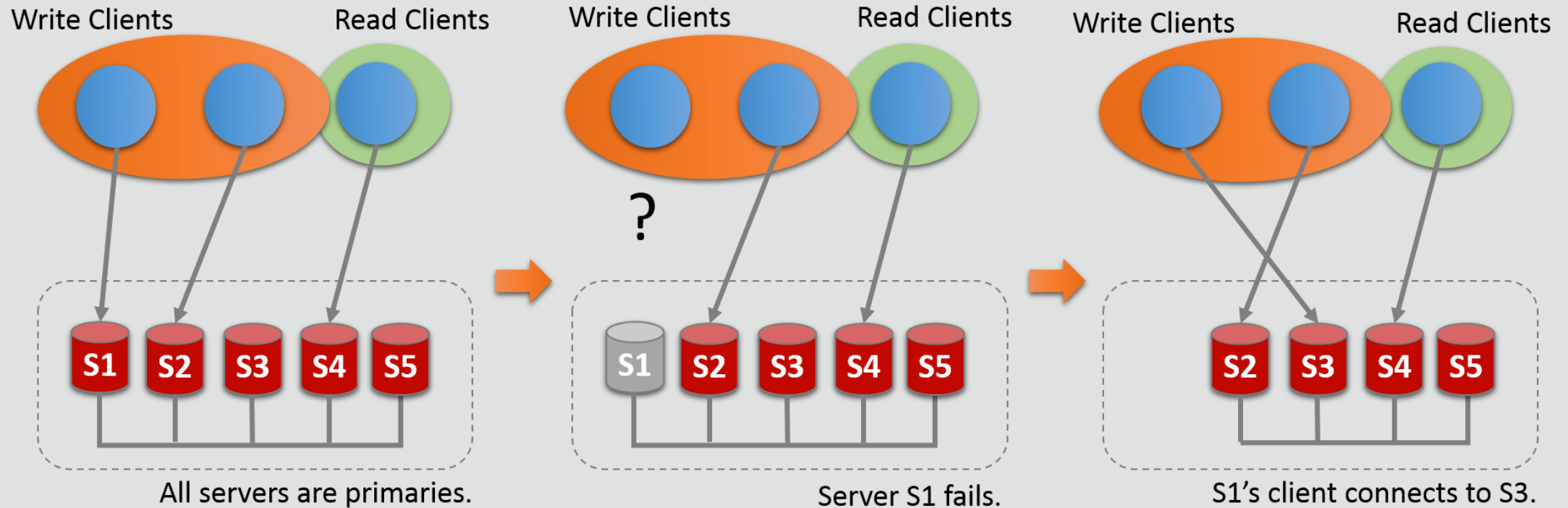
Group Replication Server States

STATUS	DESCRIPTION	GROUP SYNCHRONIZED
ONLINE	The server is an active member of a group and in a fully functioning state	Y
RECOVERING	The server has joined a group and is in the process of becoming an active member. Distributed recovery is currently taking place using a remote cloning operation or binary log.	N
OFFLINE	The Group Replication plugin is loaded but the member does not belong to any group	N
ERROR	The member is in an error state and is not functioning correctly as a group member	N
UNREACHABLE	The local failure detector suspects that the member cannot be contacted, because the group's messages are timing out	N

Single-Primary Mode



Multi-Primary Mode



Important Notes for **multi primary**

- Optimistic execution: transactions may abort on COMMIT due to conflicts with concurrent transactions on other members.
- GR is not aware of Table locks and Named Locks
- Concurrent DDL and DDL+DML executing against the same object but on different servers is not supported
- Foreign Keys with Cascading constraints are not fully supported



Stop and Start

- Single node
 - STOP: connect to the node in sql mode and use “**stop group_replication**”
 - START: connect to the node in sql mode and use “**start group_replication**”
 - By default a node automatically rejoin to cluster when started/rebooted
- Full cluster
 - STOP
 - Stop application/Routers
 - Stop each node one by one, starting from secondaries (normal shutdown)
 - START
 - Start all instances like single instances
 - With MySQL Shell connect to **primary node** in command mode
 - Start cluster with **dba.rebootClusterFromCompleteOutage()**
 - Verify cluster status
 - Start application/routers

InnoDB Cluster upgrade

1. Upgrade MySQL Shell package
2. Start MySQL Shell, and run **dba . upgradeMetadata ()**
 - Pause when the prompt is displayed
 - Use The DryRun option to check upfront
3. Rolling upgrade MySQL Routers
4. Resume dba.upgradeMetadata() execution
5. Upgrade MySQL Servers
 - Start from secondary to primary
 - Wait that the upgraded node is online before upgrade the new one

InnoDB Cluster finals

- MySQL Shell save metadata for InnoDB Cluster in the database `mysql_innodb_cluster_metadata`
 - Metadata are replicated on every node
- The users used to manage replication are automatically generated
 - `mysql_innodb_cluster_...`
 - `mysql_router1_...`
 - Use `<cluster>.resetRecoveryAccountsPassword()` to reset the passwords for the internal recovery accounts created by InnoDB Cluster
- Administrative user for both cluster and router can be created and managed with MySQL Shell
 - `dba.configureInstance()`, `<cluster>.setupRouterAccount()` and `<cluster>.setupAdminAccount()`

InnoDB Cluster minimal setup

Step 1 of 2. Configure Instances

1. Check each instance configuration

```
e.g. MySQL JS > dba.checkInstanceConfiguration('...')
```

2. If not ok, prepare instances for group replication

```
e.g. MySQL JS > dba.configureInstance('...')
```

3. connect to first instance (the primary)

```
e.g. MySQL JS > \c root@mysql1
```

4. create cluster

```
e.g. MySQL JS > var cluster=dba.createCluster('mycluster');
```

5. add all other nodes with cluster method

```
e.g. MySQL JS > cluster.addInstance('root@mysql2')
```

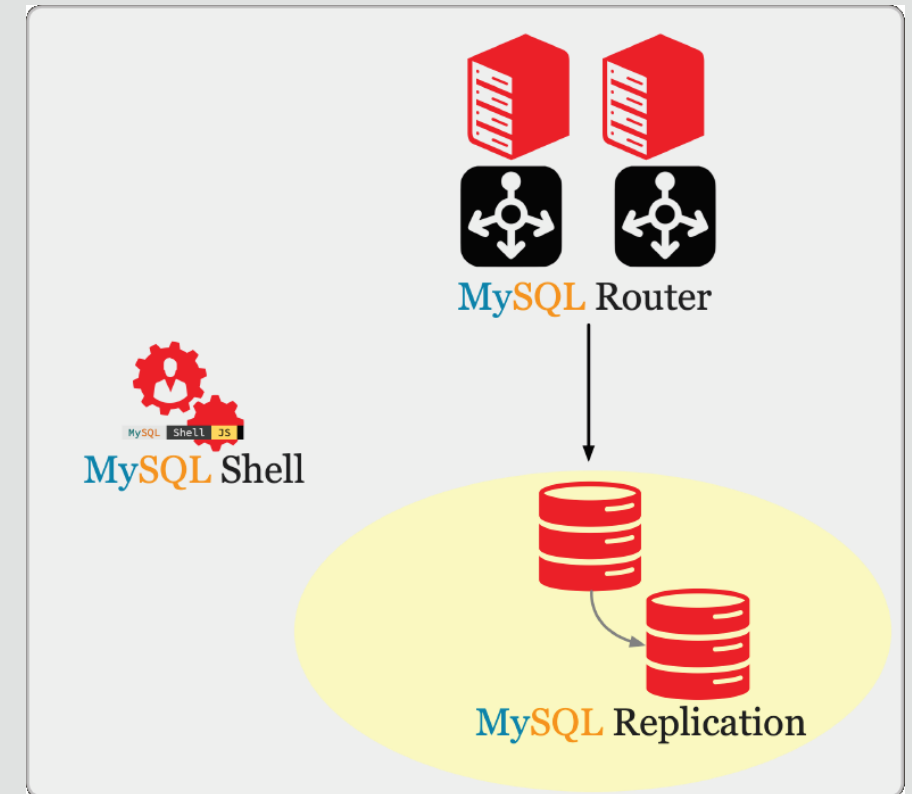
That's it!

Now you can check cluster status with cluster method

```
e.g. MySQL JS > cluster.status()
```

MySQL Replicaset (8.0.19+)

- 'classic', 'asynchronous' Replication based Solution, fully integrated
- Integrated MySQL Router
 - Automatic Routing
 - (asynchronous) Read Scale out
- Ease of use with MySQL Shell
 - Configuring, Adding, Removing members
 - Automatic Member Provisioning (CLONE)
- Switchover & Failover (manual)



Creating a ReplicaSet - AdminAPI

1. Configure Instances for ReplicaSet usage

```
MySQL JS > dba.configureReplicaSetInstance('root@mysql1')
```

2. Create the ReplicaSet

```
MySQL JS > \c root@mysql1:3306
```

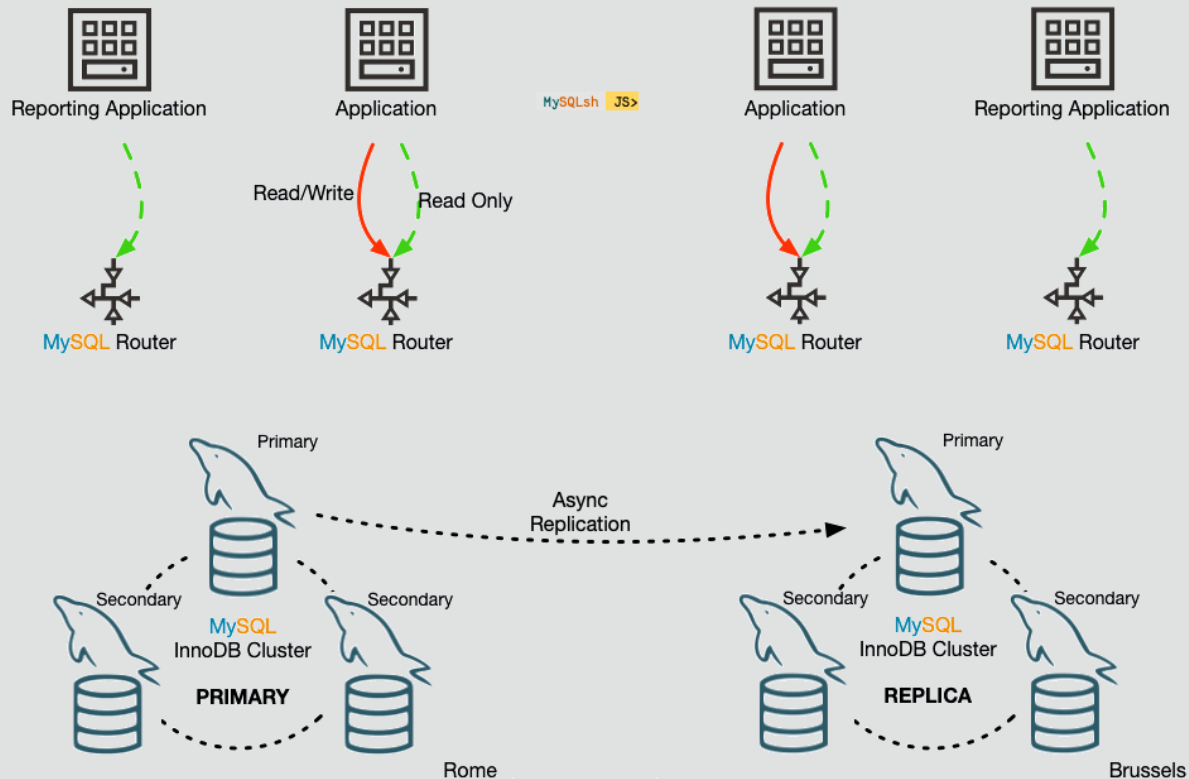
```
MySQL JS > replicaset = dba.createReplicaSet('myReplicaSet')
```

3. Add Instances to the ReplicaSet

```
MySQL JS > replicaset.addInstance('mysql2')
```

MySQL InnoDB ClusterSet

Replication between InnoDB Clusters



High Availability (Failure Within a Region)

- RPO = 0
- RTO = seconds (automatic failover)

Disaster Recovery (Region Failure)

- RPO != 0
- RTO = minutes or more (manual failover)
- No write performance impact

Features

- Easy to use
- Add/remove nodes/clusters online
- No need to reconfigure application if the topology changes

Creating a ClusterSet - AdminAPI

1. Create a regular MySQL InnoDB Cluster
2. Create the ClusterSet

```
MySQL JS > clusterset = bru.createClusterSet('clusterset')
```

2. Create the replica cluster

```
MySQL JS > repl = clusterset.createReplicaCluster('mysql1-dr', 'REPL_CLU')
```

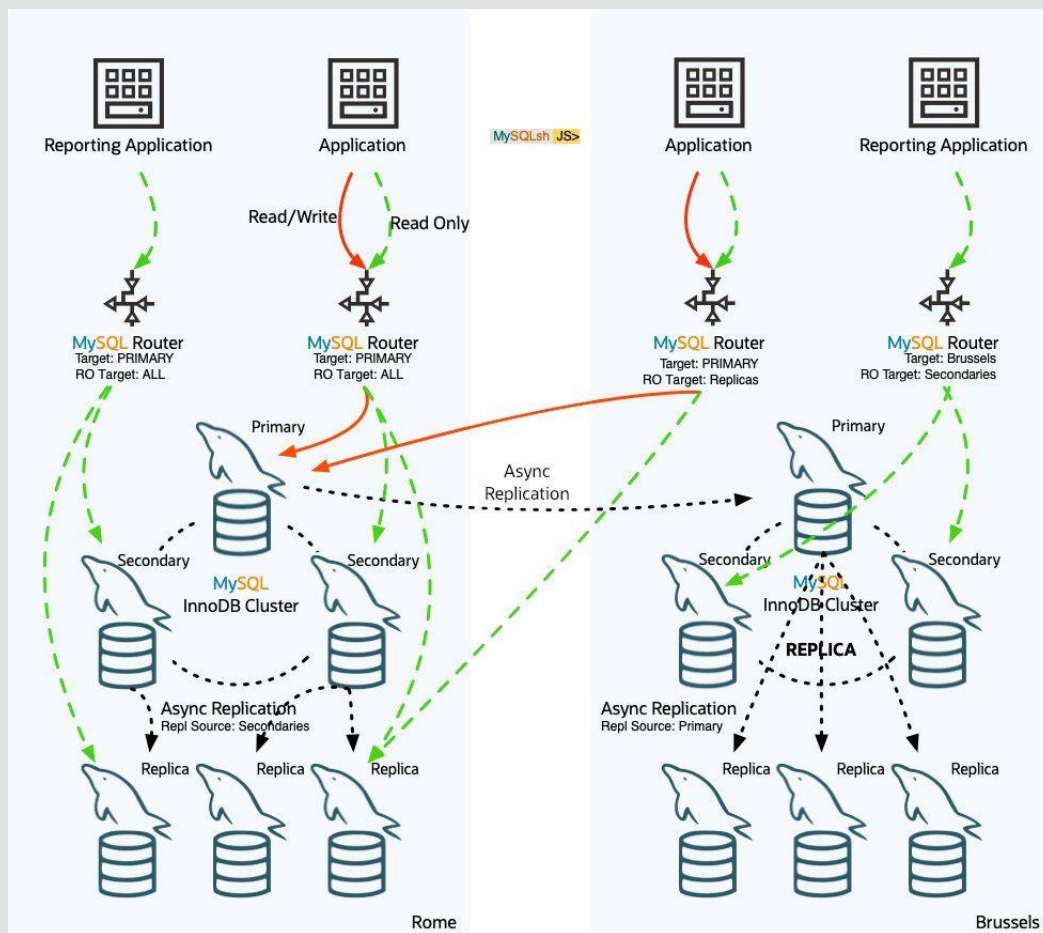
```
MySQL JS > repl.addInstance('root@mysql2-dr')
```

3. Check ClusterSet status

```
MySQL JS > Clusterset.status()
```


MySQL InnoDB Read Replicas

Improved scalability and redundancy



Read Replicas

- read-only copies of a Cluster member

Scalability

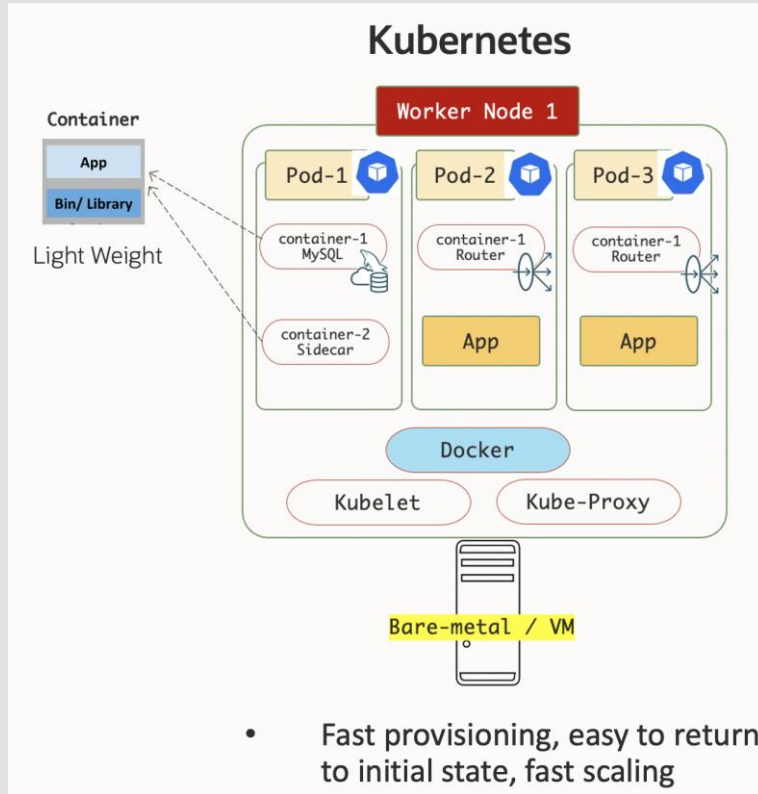
- Asynchronous replication enable to scale out workload, offload read requests from your Cluster to one, or more, dedicated read-only instances

Additional redundancy

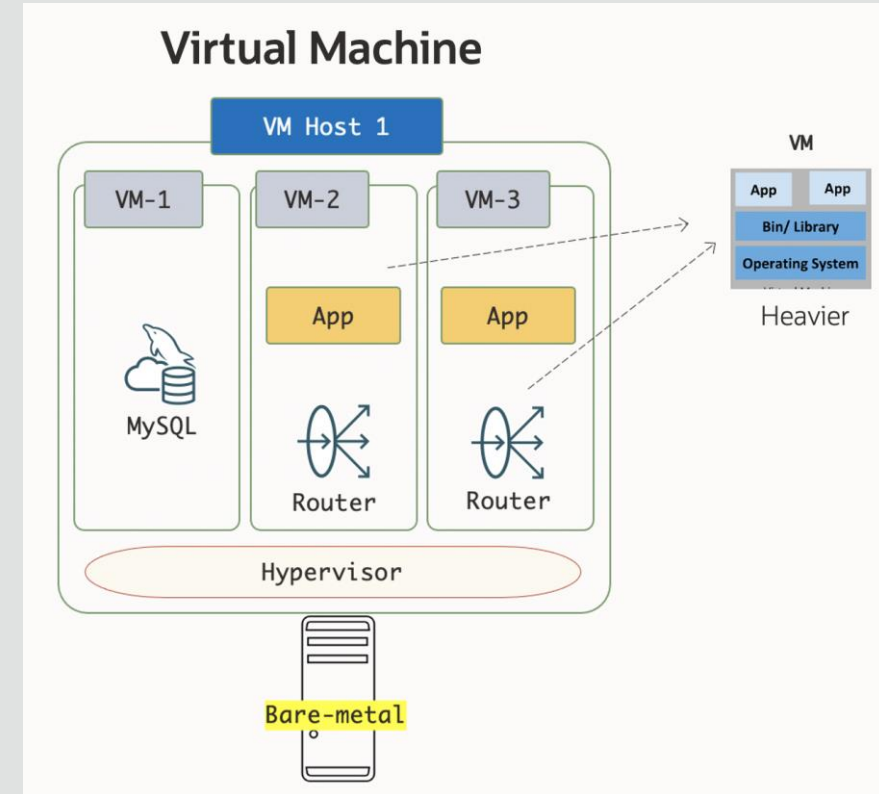
- Read replicas can be promoted to cluster members

Fully managed with MySQL Shell

Kubernetes Vs. Virtual Machine (VM)



Vs.



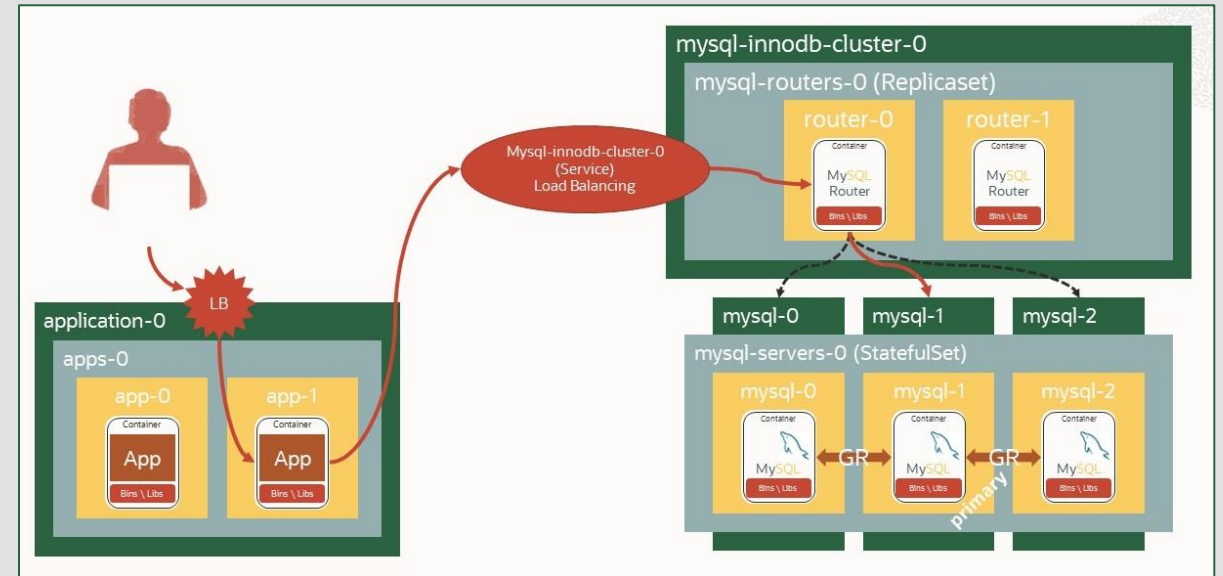
- A portable, extensible, open-source platform for managing containerized workloads and services, that facilitates both declarative configuration and automation.
- The desired state and other definitions are defined in a manifest (yaml file)

MySQL and Kubernetes

- MySQL single instance can be easily deployed with Kubernetes features
- A database cluster is not easy like a single instance
 - A cluster contains multiple nodes
 - Specific sequences in nodes startup/shutdown
 - Specific sequences in cluster updates
 - Create multiple yaml files for various resources (StatefulSet, ReplicaSets, Services...)
 - Scale out/Scale in have requirements
 - Rolling update has requirements
- An operator offer an easy interface to create and manage MySQL clusters where coordination between multiple resources is required (MySQL Instances, MySQL Router, pods, deployments, replica sets, services...)

MySQL Operator for Kubernetes

- The MySQL Operator for Kubernetes (since 8.0.29) is designed to operate a MySQL InnoDB Cluster in Kubernetes
- Automated deployment and management of MySQL Server & MySQL Routers
- Self-healing
- Backup & Restore
- Rolling upgrades with minimal downtime
- Read Scaling
 - Add/remove members as needed
 - Configurable consistency levels
- Helm support
- Developed and supported by the MySQL Team





Hands-On Labs



7a. MySQL InnoDB Cluster



ORACLE